Reflection

Personal Portfolio 4:

Skatepark Enschede in SkaterXL

https://www.youtube.com/watch?v=Uk_ifSwdao4

Emily Murad

523058

Table of Contents

Learning outcomes **3**
Learning outcomes 1: The process 4
Learning outcomes 2: The process 5
Reflection of the process **11**
What went well? 11
What are the main reasons for this success? 11
What can I do to force these reasons to occur again in the future? 11
What didn't go so well? 11
What is the cause of this? 11
What can I do to prevent this cause in the future? 12
Hour registration **13**
Reflection after the process **14**
Did you complete your learning goals? 14
What do you want to work on next term? 14
Future learning outcomes? 14
References **15**

Learning outcomes

Learning outcome 1:
As an engineer student I want to learn the theory of how to turn a 3d skatepark model in to a SkaterXL map to prepare myself for doing it with the skatepark model I made in personal porfolio 3 with the end goal being: Learn the theory of how to make maps for the game SkaterXL.

Learning outcome 2:
As an engineer student I want to create a skatepark map for the game SkaterXL with the 3d skatepark model I made for personal porfolio 3 with the end goal being be able to turn a 3d model into a SkaterXL map that can be used in game.

Learning outcome 1: The process

I started by researching how maps are made in SkaterXL. Most of the material I could find was outdated, with the latest sources being in 2020. But this did not discourage me, even though the SkaterXL modding community wasn't as lively anymore. I found a GitHub modding guide (SkaterXLModding, 2020) that had a lot of information that even though beginner friendly, I did not seem to understand much of. So I moved on to YouTube where I ended up finding there are multiple ways to make maps. I specifically found multiple ways to "spline" obstacles. This is the process of making the obstacles grindable, which means have the skateboard stick to the obstacles to do tricks in the game. During my search there were barely sources, I found different videos by a YouTuber called SqueegeeDinoToy (SqueegeeDinoToy, n.d.).

The first video (SqueegeeDinoToy, Skater XL Mapping Guides - Creating a Map, 2020) and way to create maps was one where you do it fully through Unity. You create objects and then assign the correct tags and layers for them, exporting it to the game. However, I created my own skatepark in Blender previously, and I want to be using this map instead. So I went on and found another video (SqueegeeDinoToy, SkaterXL Mapping Tutorials | Blender to SXL Map, 2020) showing how you can spline models from Blender in Unity by manually creating colliders for the obstacles.

And at last I found another video (SqueegeeDinoToy, SkaterXL Mapping Tutorial | Effective Grindsplines w/o Dreamteck, 2020) where SqueegeeDinoToy made a spline plugin for Blender (SqueegeeDinoToy, Skater-xl-blender, 2020) where you can create grind points, the positions of the beginning and end of grindable objects. Alongside this plugin he promotes two Unity packages made by Billowper, the first automates the creation of grind colliders in Unity (Billowper, Skater-XL-Resources, 2020). The second lets you export the Unity scene as a map file with version numbering and automatically adds it to your SkaterXL mods map folder (Billowper, Skater-XL-Resources, 2020). These tools are extracted from Billowpers broader SXL tools by SqueegeeDinoToy in order to simplify what you need. After learning all this information I started by creating a Unity project, this was only the beginning of a long trial and error process as I ended up researching a lot more later down the line during the modding process.

Learning outcome 2: The process

I started out by installing the Unity version SqueegeeDinoToy uses in this video (SqueegeeDinoToy, SkaterXL Mapping Tutorial | Effective Grindsplines w/o Dreamteck, 2020). The version is 2018.4.12f1 and the moment I opened the project I received a bunch of project errors regarding DLL's. After realizing it might just be that specific version I decided to get the latest LTS of 2018.4 which ended up having the similar DLL errors upon opening the project. I attempted to resolve the issues with ChatGPT but it seemed to be no help. Later on I decided to get the latest LTS version available, Unity 6. This project opened without any issues so I decided to continue from here.

I opened my 3D model in Blender and added the plugin made by SqueegeeDinoToy (SqueegeeDinoToy, Skater-xl-blender, 2020). I decided to start simple with a test run following along the tutorial where he splines a simple round rail. I choose my ground rail and select the edge that I want to be the spline and generate the grind points. Soon I ran into the issue of the grind points orientation being different from the mesh (figure 1).

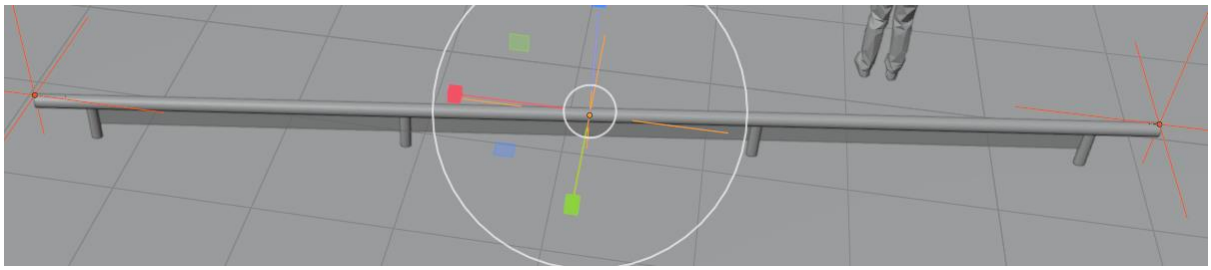


Figure 1 Grind points are rotated along world axis rather than local axis.

It took a while to figure out a solution, but it was a rather simple one. I had to rotate the mesh to be aligned with the world axis and then generate the splines. After that I had to rotate the mesh while in object mode back to its original position, this would make sure the grind points rotate along as well.

Afterwards I exported the ground rail as FBX and imported it into Unity for the next step. I follow along the video tutorial and add all required tags and layers to Unity and I use Billowper's tool (Billowper, Skater-XL-Resources, 2020) to automatically generate colliders with correct tags and layers for the ground rail. Afterwards I use Billowper's SXL tool (Billowper, Skater-XL-Resources, 2020) that lets me export the scene as a SXL map. After loading up the game and the map I quickly found out that I spawn in an empty blue void with the obstacles nowhere to be found.

So I returned to researching what the possible cause could be. I found a comment under one of SqueegeeDinoToy's older video's (SqueegeeDinoToy, SkaterXL Mapping Tutorials | Blender to SXL Map, 2020) where someone commented a fix to the blue void issue (figure 2).

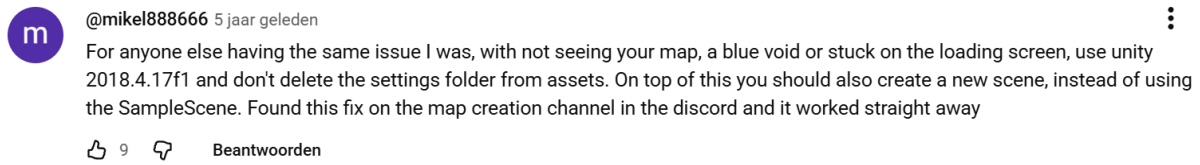


Figure 2 Someone offering a fix for the same issues I was experiencing.

So I attempted to create a new scene instead of Unity's sample scene and tried to set a spawn point as I saw elsewhere that that also could be an issue in hopes to avoid what seemed to be the clear issue, the Unity version. The attempts deemed unsuccessful, so I realized I must try another Unity version yet again. Not 2018.4.12f1 or the latest LTS of 2018.4 but rather 2018.4.17f1. So I went and installed that version and ran into the same project DLL issues again at startup.

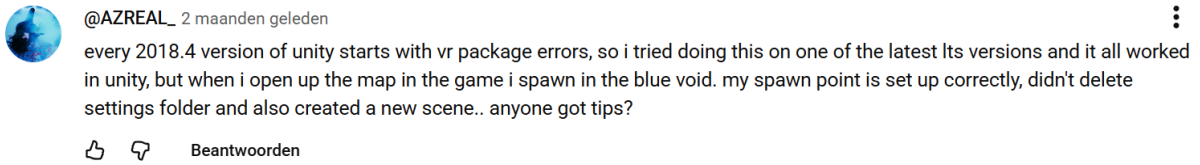


Figure 3 My attempt at requesting help unfortunately going unanswered.

This is the moment where I decided to switch PC's. I installed the version on my laptop instead of my desktop and it ended up opening without errors. So I ended up having to get Blender and setup the spline plugin and spline the ground rail again, followed by adding the Unity packages and generating colliders. However, in this version Billowper's collider generator was not working properly. Whenever I tried to generate a collider I would receive an error.

I tried to contact SqueegeeDinoToy through discord but it was no help, I also found different SXL map creators their discord servers and tried asking there or even directly direct messaging them but still there was no real help. I was feeling pretty hopeless at this point and almost considered giving up and changing my learning goals for the subject.

Later I decided to not yet give up and I did a deep dive in to Billowper's tools as SqueegeeDinotoy had previously mentioned that he extracted the specific tools from a broader toolset. I went on and found Billowper's full toolset (Billowper, SXL-Map-Tools, 2020). After going more down the Billowper rabbit hole, aka his GitHub, I found out that he had a sample Unity project repository (Billowper, SXL-Template-Project, 2020) that shows how you can use his toolset. The version he uses in the template project is 2019.3.15f1 and so I downloaded yet another Unity version.

I decided to simply add my skatepark to his template project and continue on from there. Soon I found out that in his template project the tools were fully broken. I kept receiving errors similar, but also many more than the previous attempt in Unity version 2018.4.17f1. After this setback I decided to create a brand new project in the same version Billowper used for his template project, Unity version 2019.3.15f1, and then added the full toolset package manually. After going through the motions generating the colliders for my rail, exporting the scene to a map file, and running it in the game… I found that finally, it works!

It took quite some time and trial and error on this part in order to get things working, but it was very rewarding in the end. Now the real fun could start.

Soon I realized the scale of everything was way too big compared to the skater in the game. As a quick fix I decided to scale the model down through Unity, even though I was aware I would have to fix it later on in Blender for the best results.

I ran in to many issues regarding the 3D model. I had many faces where the normals were flipped (figure 4). So I had to go back in to Blender to flip them, or for some I added a solidify modifier in order for them to have normals and thus faces in both directions (figure 5).

I also learned that I should not apply all transforms before exporting as that placed all grind points will be at the world origin.

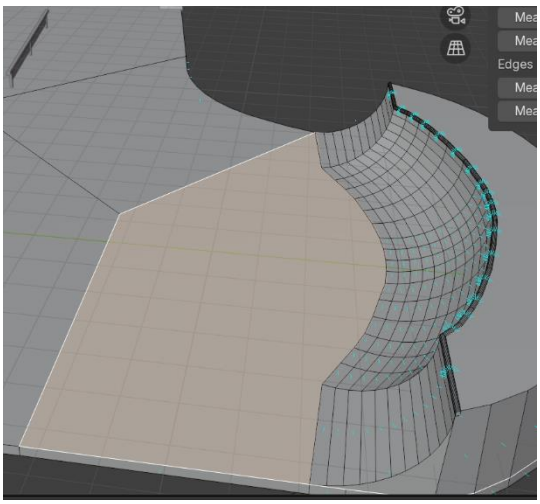


Figure 4 Selected area's normals are flipped.

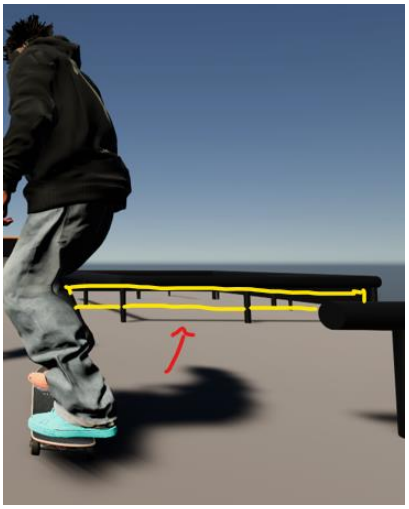


Figure 5 Missing faces (yellow area).

I also changed the scale of the ground rail manually and realized that while scaling the meshes also rotate, which was a bit hard to get through. During this point I also realized that the rail has 4 legs instead of the 3 I had modeled before, so I added another leg (figure 6).

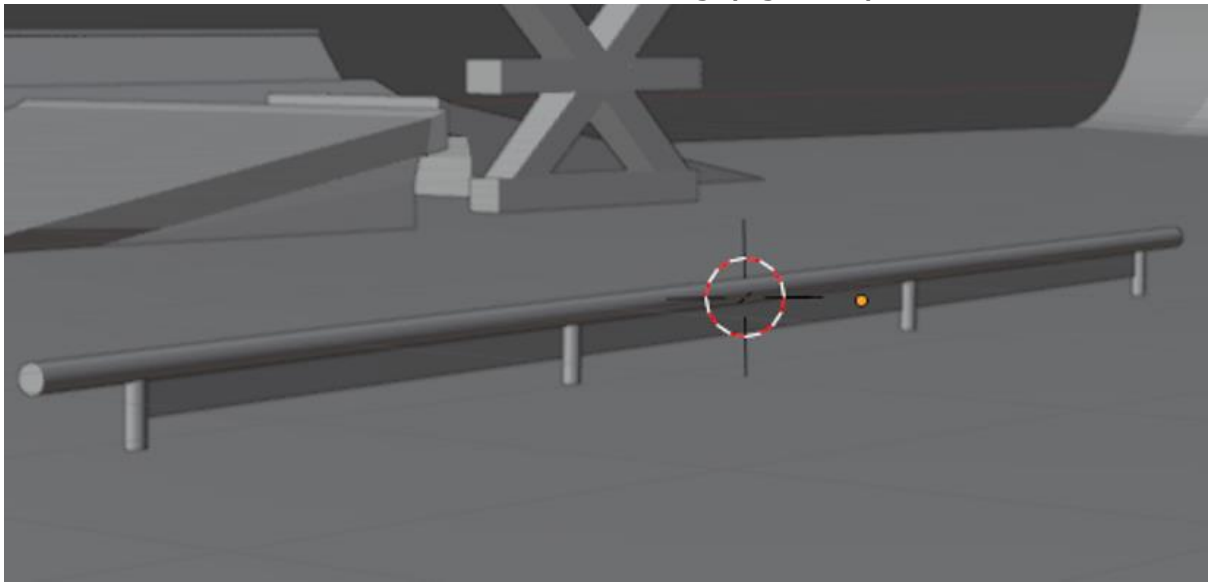


Figure 6 Ground rail remodeled, 4 legs and different scale.

After scaling the rail I realized I should scale everything down since it was too big for the gameplay. I ended up scaling the whole skatepark down a bit. This had some consequences that made me reposition some obstacles such as this quarter bank (figure 7) but also other ground obtacles.

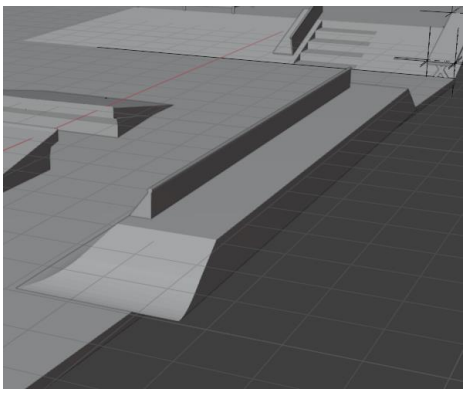


Figure 7 Sticking out due to scaling.

After this I also started splining obstacles that might behave weirdly, such as angled or rounded obstacles. So I splined the bent rail, hubba ledge and square rail in order to test them. Afterwards in Unity I found that there are some quirks to these obstacles. For example, the bent rail's grind points have moved (figure 8) even though I did not apply transform at exporting. So I had to figure out a way to keep them at their original Blender position. Which ended up being applying transform for the specific object before creating splines, but not applying transforms at export.

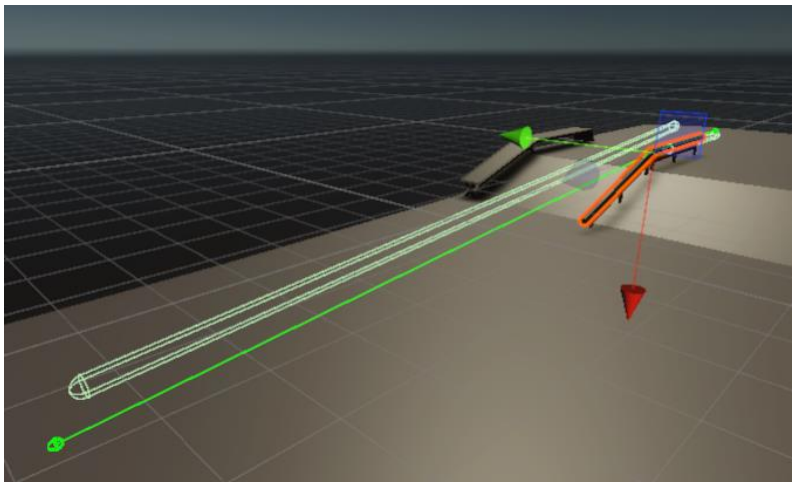


Figure 8 Incorrect grind points positions causing weird colliders.

I also realized that the squared rails called for extremely weird grind colliders at generation due to their squared shape not being point A to B but rather A to B to C to D (figure 9).

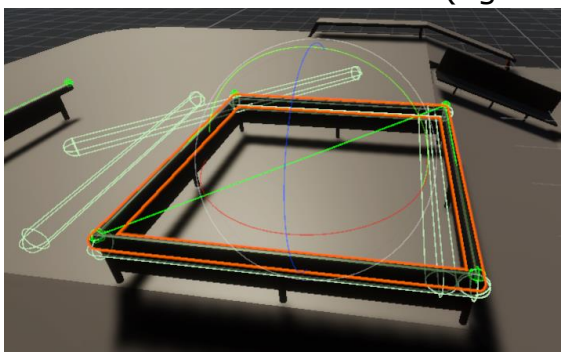


Figure 9 Square rail colliders weird behaviour.

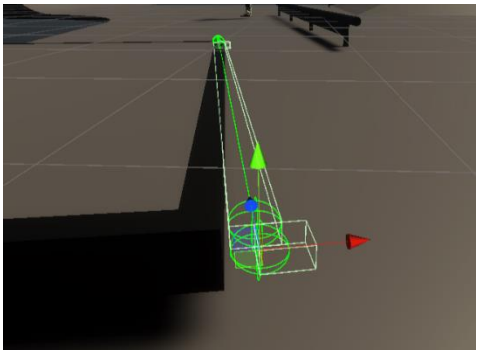


Figure 10 Grind points are aligned to world space.

I tried to fix the grind points separately in Blender but that seemed no help. Since apparently, I could not move grind points in Blender without moving the vertices as well. The only fix was to unpack the FBX prefab in Unity so I can manually move the grind points myself and generate the colliders afterwards. And still I ended up with 4 colliders too many, so I deleted them. I also noticed that the ground points of all angled obstacles were off (figure 10) because I forgot to apply transform in object mode before creating the splines, which caused me to return to Blender yet again.

As I was splining the Enschede logo I realized that it has many vertices that kept making the whole surface collider weird, it would be weird differences in height between the bunch of colliders it created. So I decided to ease the process by simply creating an edge across the whole top since the whole of it has to be grindable anyway. This discards the worrying about making it an edge with the exact right widths left and right with a bunch of colliders. Now it can be one simple collider instead (figure 11).

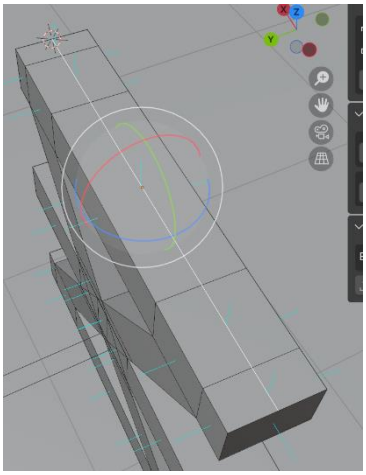


Figure 11 Single subdivision for the spline.

Afterwards I realized the banks had weird colliders generated even though the grind points were correct (figure 12), I ended up manually aligning them for ease as I had already went through majority of generating colliders for the whole park and it would take a while to redo all of it again.

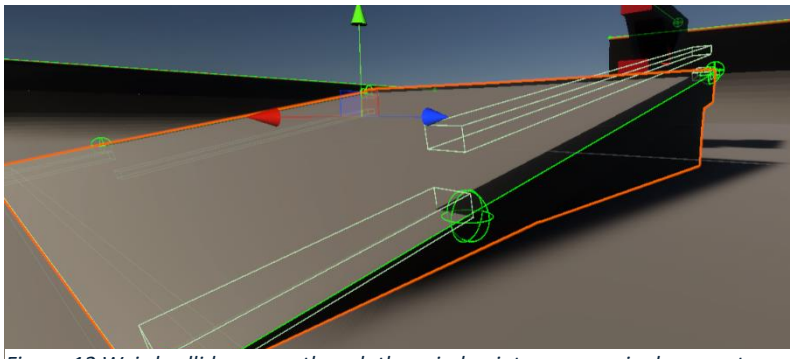


Figure 12 Weird colliders even though the grind points are seemingly correct.

Here are some extra images from the process where I remodeled and scaled the vert part (figure 13), splined the bent rail despite its odd shape (figure 14), SqueegeeDinoToy's Blender splining tool (figure 15), Billowper's Unity collider generation tool (figure 16/17), Billowper's Unity map export tool (figure 18), and the whole map splined (figure 19).

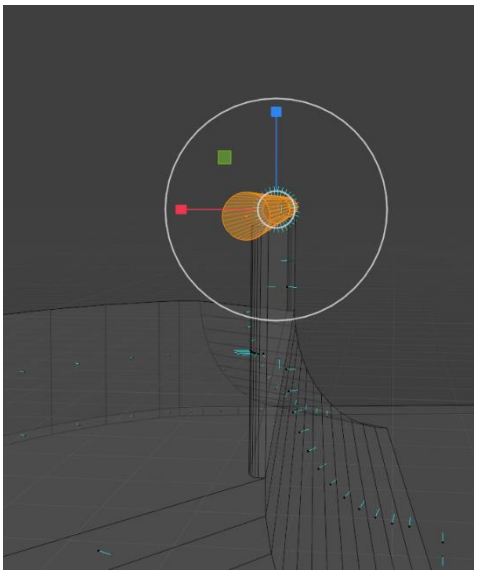


Figure 13 Remodeling and scaling of vert part.

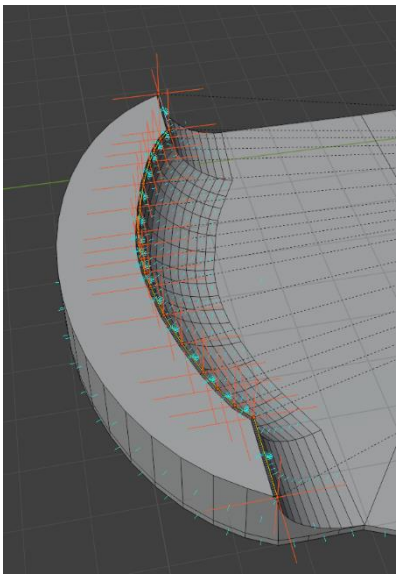


Figure 14 Splining the bent rail.

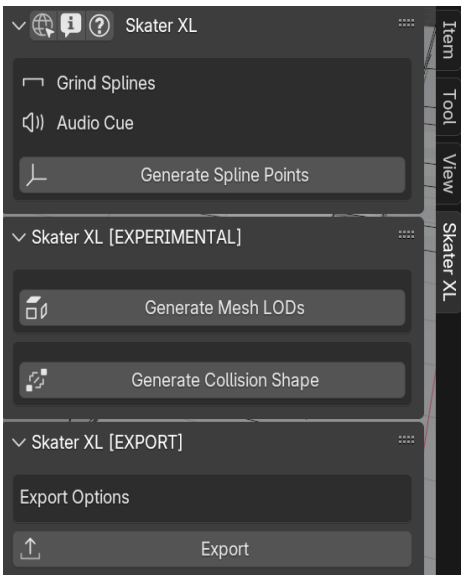


Figure 15 Blender spline tool.

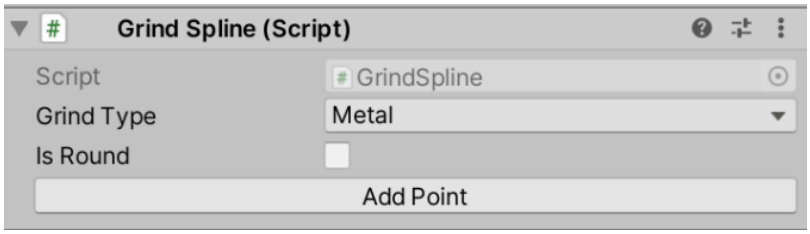


Figure 16 Unity spline tool (assigned to grindpoints parent object.

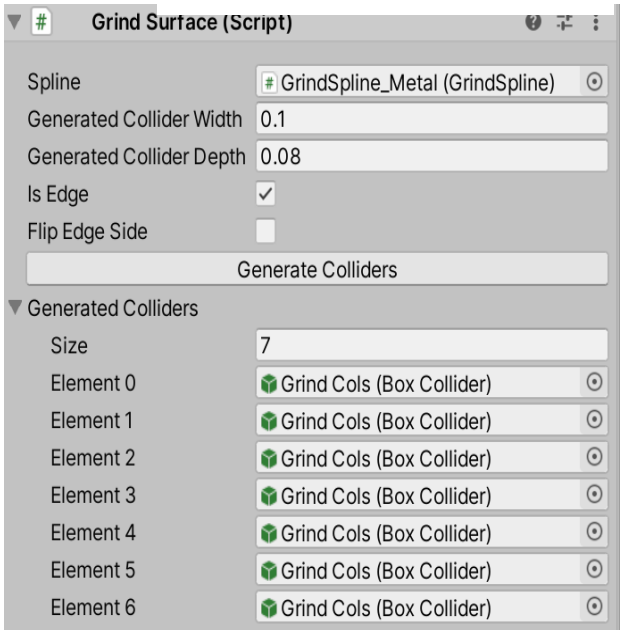


Figure 17 Unity spline tool (assigned to obstacle obect).

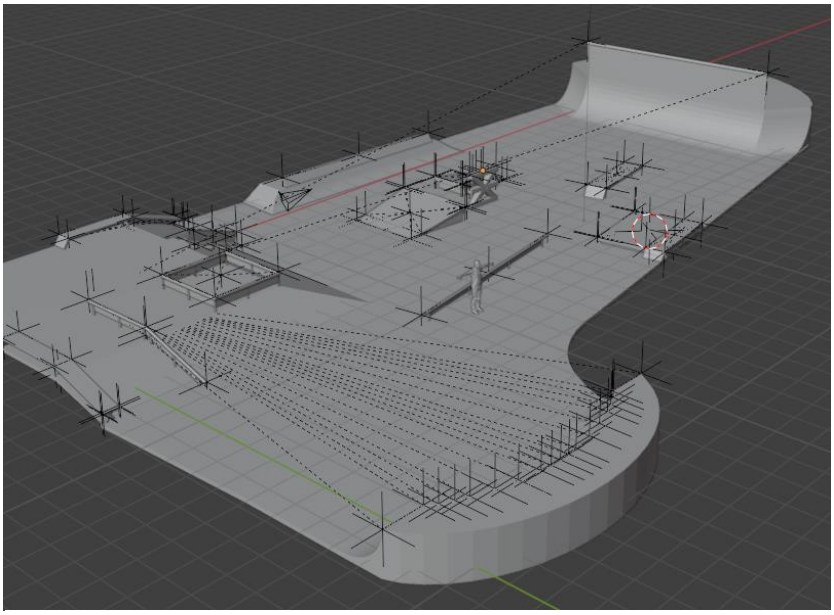


Figure 19 Final fully splined model in Blender.

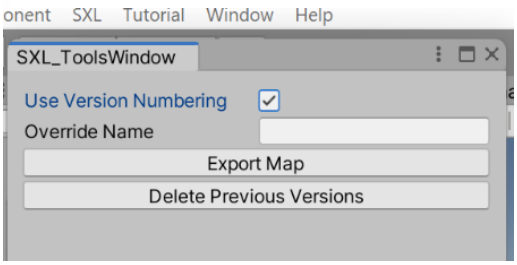


Figure 18 Map export tool with version numbering.

Reflection of the process

What went well?

Research, it didn't feel like it was going well at the beginning but looking back I managed to get to the point I did because of it. Also splining in Blender went great even though I had to circle back a bunch, the process was quite smooth. Just like generating colliders in Unity, I believe that went pretty well as the result was a working skate map! Overall the self-study part is something that went very well, even when odds were against me and reaching out for help proved no results.

What are the main reasons for this success?

My motivation and persistence were great factors. I don't think I would have kept the search for a solution otherwise. The want to do something and succeed at it. I always wanted to make a Skater XL map ever since I started playing the game and even tried to right before joining CMGT, so this was like a small dream that I just needed to achieve, with all the knowledge I gained from the study I feel like I could actually do it.

What can I do to force these reasons to occur again in the future?

I believe motivation is something you can create yourself, if you really want to achieve something you can muster up excuses to keep going and trying until you get there. If others have done it (creating maps) before, so can I. So I believe I should find something or anything that could spark motivation within me for whatever it is that I'm trying to accomplish.

What didn't go so well?

Other than the rough start, I spent too much time going back and forth between Blender and Unity on small details that could have been avoided. While trial and error is a good process, I believe that what I worked on could have been more efficiently done. I ended up having 26 versions of the map until I had one that fully worked, not bug-less, but definitely a good result I'm proud of. A bunch of the iterations done here could have been prevented.

What is the cause of this?

The lack of information out there about SkaterXL mapping since it's already quite a niche game, let alone topic to be making maps for said game. Information was outdated and most people within the community have been radio silent whenever I tried to reach out for help in Discord communities or direct messaging. Furthermore, a bunch of the issues I ran in to had to do with orientations in Blender. Unapplied transformations, flipped normals, and weird rotations were recurring problems that could have been entirely avoided.

What can I do to prevent these things in the future?

While it might be hard, still try and find active people in the community. Maybe by making public posts and being more active on social media. Also simply choosing a newer topic could be better rather than diving into old and outdated topics. For the rest I think that my lack of modeling knowledge could have prevented a lot. I believe a lot of the issues I ran in to would have been simple modeling knowledge that I should have focused on during my previous personal portfolio when I created the skatepark. And also, I could have tested multiple obstacles at once rather than one by one, as splining was easier to redo than I had expected, with worst case scenarios just being manually setting them in Unity. I need to not be so worried about just trying out things.

Hour registration

Personal Portfolio 4 - Hour registration

When	What	Hours
22/Sept	Research about Skater XL mapping / found youtube videos	3,5
29/Sept	Install unity version, required plugins and assets / ran into Unity bugs with setup / fix normals / apply grind splines to ground rail	8
02/Oct	Research and tried to fix Unity setup bugs / reinstalled different Unity versions	4
04/Oct	Tried another Unity verison that works / remake grind splines for ground rail / make colliders in unity / map test version didn't load / blue void	8,5
09/Oct	More research about SXL modding / attempted different Billowper tool version / remade colliders / still blue void when loading in game	7
24/Oct	Research Billowper's tools and found another Unity version / installed new Unity version / made colliders for ground rail / buggy but map loads and rail works	9
27/Oct	Used squeegedinotoys Blender plugin to set spline grind points / fix ground rail grind points orientation / made unity colliders and tested ground rail	3,5
03/Nov	Create spline grind points for different obstacles in order to test surfaces before fully committing to all obstacles (angled surfaces, squared rail, bent rail etc	8,5
05/Nov	Scale seemed too big for gameplay so rescaled in Blender / Remodeled ground rail to have 4 legs and correct scale / ended up rescaling everything in Blende	9,5
06/Nov	Found mesh issues / added solidify modifier in Blender / had to flip normals of a bunch of faces / remake colliders in Unity to test scale	7,5
12/Nov	Fix lighting in Unity / Created spline grind points for Hubba's, box ledge, high and low ledge	4
18/Nov	Create spline grind points for all obstacles / ran into issue of grind points orientation being different from Blender / reapply transforms and remake splines	6,5
25/Nov	Make colliders for all obstacles / manually colliders for square rails and all ledges due to recurring rotation issue	7
02/Dec	Test each obstacle in game / rescale Enschede logo in Blender after finding missing faces / record gameplay	5
10/Dec	Edit gameplay video	4,5
	Total Hours	96

Reflection after the process

Did you complete your learning goals?

Yes, I completed my goals. I can confidently confirm that I fully understand the theory of how one can create a map for SkaterXL, with multiple possible ways to crete them. I have sufficiently self-educated to the point where I was able to indeed create a map of my own. This map made with my own 3D skatepark model now officially fully working in the game SkaterXL. I can proudly say I could repeat the process for any other model in order to create SkaterXL maps.

What do you want to work on next term?

Next term I would like to dive in to Unreal Engine and try and create a simple game in there. I think it would be a great addition to my portfolio that would land me more opportunities. I already worked with blueprints briefly, but I believe creating something with C++ would be something interesting and fun to work on.

Future learning outcomes?

I’m also interested in VR / AR development. I would like to see how that works and maybe create some entertaining horror experience with that.

References

(n.d.). Retrieved from ChatGPT: https://chatgpt.com/

Billowper. (2020). *Skater-XL-Resources*. Retrieved from GitHub: https://github.com/SqueegeeDino/Skater-XL-Resources

Billowper. (2020). *Skater-XL-Resources*. Retrieved from GitHub: https://github.com/SqueegeeDino/Skater-XL-Resources

Billowper. (2020). *SXL-Map-Tools*. Retrieved from GitHub: https://github.com/billowper/SXL-Map-Tools

Billowper. (2020). *SXL-Template-Project*. Retrieved from GitHub: https://github.com/billowper/SXL-Template-Project

SkaterXLModding. (2020). *skater-xl-mapping-wiki*. Retrieved from GitHub: https://github.com/SkaterXLModding/skater-xl-mapping-wiki

SqueegeeDinoToy. (n.d.). Retrieved from YouTube: https://www.youtube.com/@SqueegeeDinoToy

SqueegeeDinoToy. (2020, 07 28). *Skater XL Mapping Guides - Creating a Map*. Retrieved from YouTube: https://www.youtube.com/watch?v=FvtyVPoweoo

SqueegeeDinoToy. (2020, 05 17). *SkaterXL Mapping Tutorial | Effective Grindsplines w/o Dreamteck*. Retrieved from YouTube: https://www.youtube.com/watch?v=EeYJRSMdsjQ

SqueegeeDinoToy. (2020, 01 06). *SkaterXL Mapping Tutorials | Blender to SXL Map*. Retrieved from YouTube: https://www.youtube.com/watch?v=lh74M2i83-s&lc=UgwF80T83VboG-rNTFx4AaABAg

SqueegeeDinoToy. (2020). *Skater-xl-blender*. Retrieved from GitHub: https://github.com/SkaterXLModding/skater-xl-blender